Pi-hole DNS Sinkhole Home Lab

Sam Suther | April 2026

Overview

I set up a Raspberry Pi 5 as a network-wide DNS sinkhole using Pi-hole. The short version: instead of your router handling DNS normally, you point everything at the Pi. It filters out ad domains and trackers before they ever load, and every device on the network benefits from it automatically. I wanted hands-on experience with Linux, DNS, and real network configuration. Reading about this stuff is one thing. Actually setting it up, troubleshooting it, and breaking it a few times is something else.

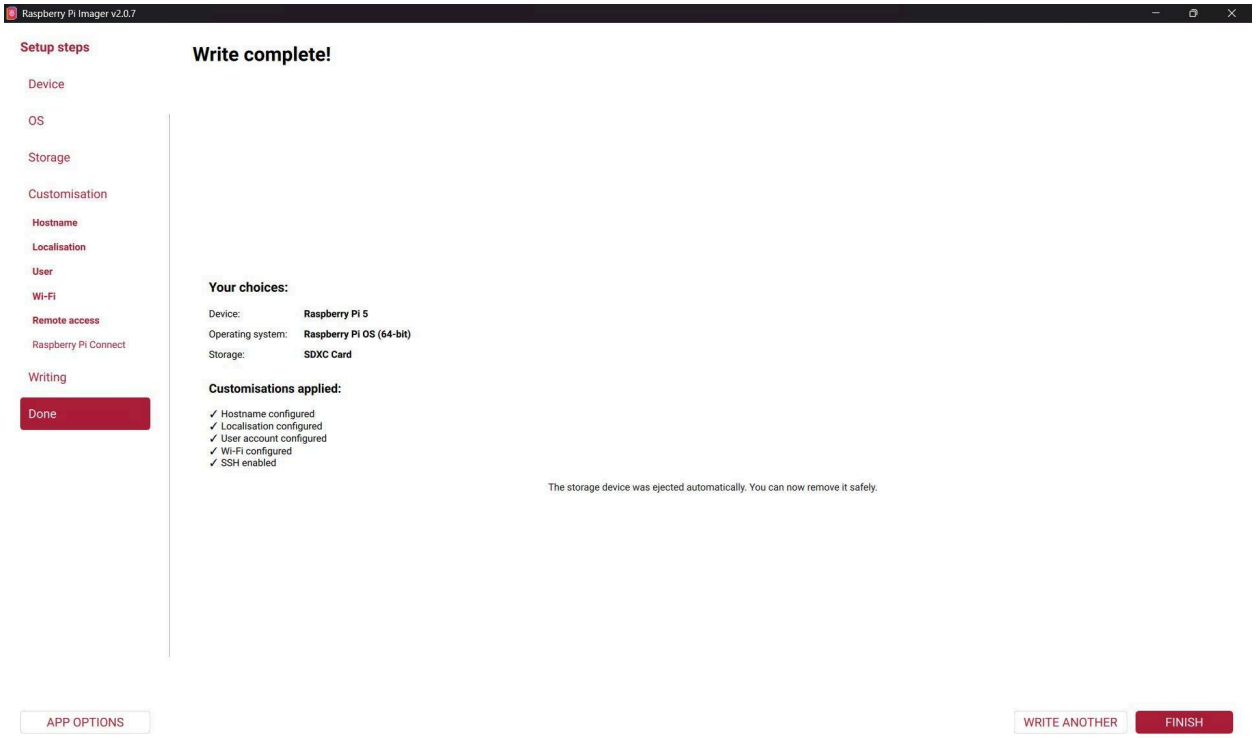
Hardware and Tools

- Raspberry Pi 5
- Micro SD card running Raspberry Pi OS Lite (64-bit)
- Raspberry Pi USB-C power supply
- Raspberry Pi Imager
- Verizon CR200A router
- Windows laptop for SSH access
- Pi-hole v6

Setup

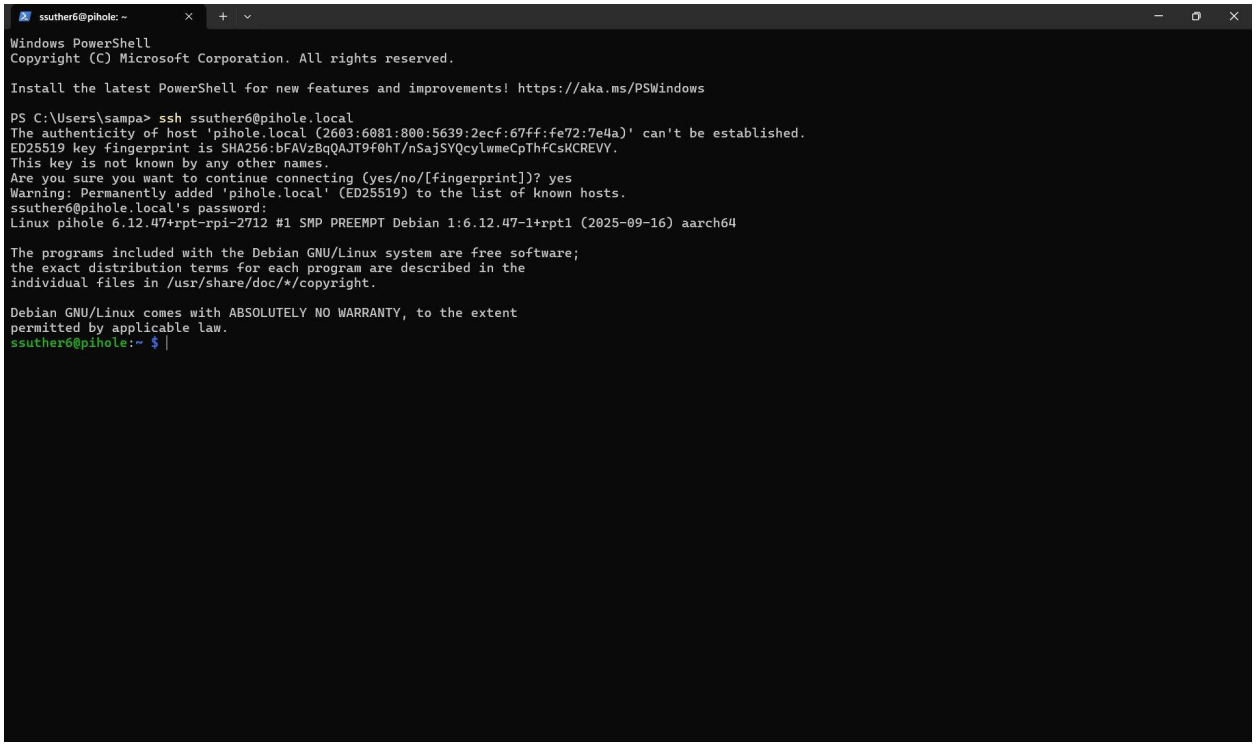
Flashing the SD Card

I used Raspberry Pi Imager to write Raspberry Pi OS Lite onto the SD card. Before flashing, there are a few things to configure in the OS customization settings: hostname, user account, WiFi credentials, and SSH. Getting SSH enabled here is what lets you control the Pi entirely from a laptop with no monitor or keyboard attached. Once that was done, I wrote the image and waited for it to verify.



Connecting via SSH

After the Pi booted and connected to WiFi, I SSH'd in from PowerShell on my laptop. The first connection will show a warning about the host key not being recognized. That is normal and expected. You type yes, enter your password, and you are in.



Installing Pi-hole

After updating the system, I ran the Pi-hole installer with one command:

```
curl -sSL https://install.pi-hole.net | bash
```

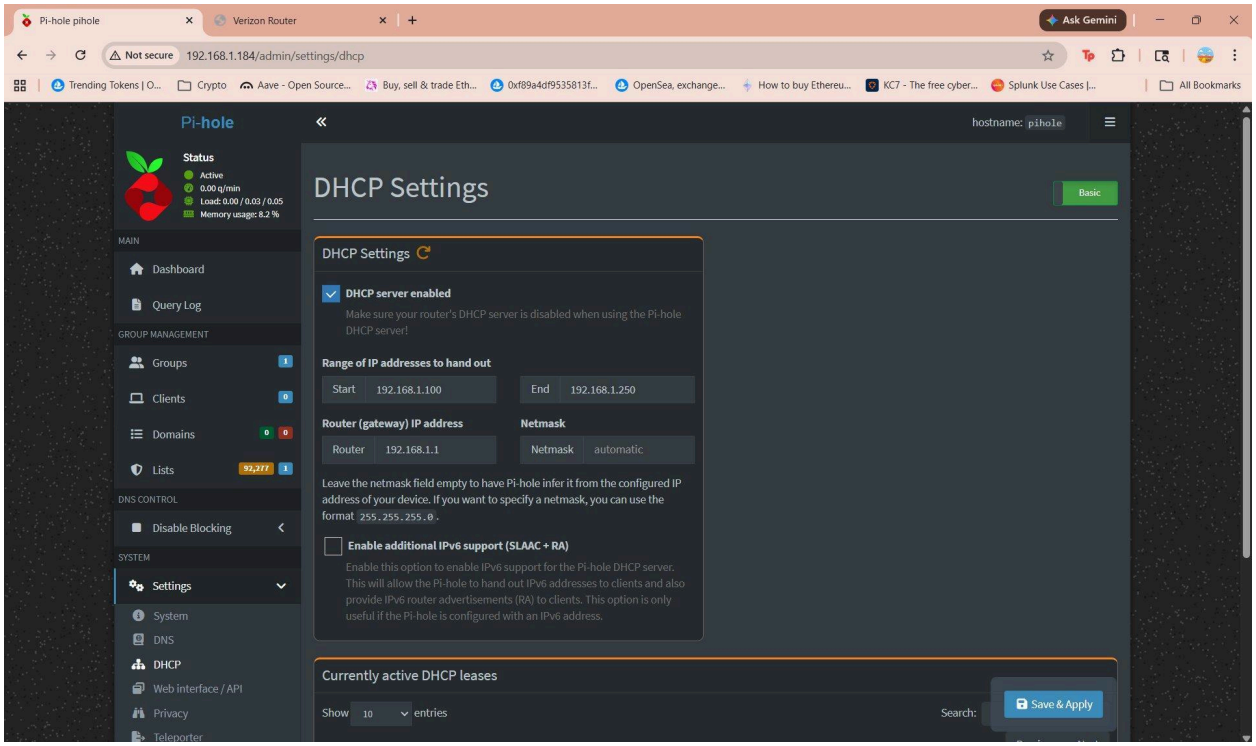
The installer runs a text-based setup screen that walks through the configuration. I chose wlan0 as the network interface, Google as the upstream DNS provider, and accepted the default blocklist. When it finished, the terminal printed the web interface URL and a generated admin password.

```
ssuther6@pihole: ~
  [i] Number of gravity domains: 92277 (92277 unique domains)
  [i] Number of antigravity domains: 0 (0 unique domains)
  [i] Number of exact denied domains: 0
  [i] Number of regex denied filters: 0
  [i] Number of exact allowed domains: 0
  [i] Number of regex allowed filters: 0

  [✓] Optimizing database
  [✓] Swapping databases
  [✓] The old database remains available
  [✓] Cleaning up stray matter

  [✓] Done.

  [i] You may now configure your devices to use the Pi-hole as their DNS server
  [i] Pi-hole DNS (IPv4): 192.168.1.184
  [i] Pi-hole DNS (IPv6): 2600:1004:a033:9f84:2ecf:67ff:fe72:7e4a
  [i] If you have not done so already, the above IP should be set to static.
  [i] View the web interface at http://pi.hole:80/admin or http://192.168.1.184:80/admin

  [i] Web Interface password:
  [i] This can be changed using 'pihole setpassword'

  [i] To allow your user to use all CLI functions without authentication, refer to
    our documentation at: https://docs.pi-hole.net/main/post-install/


  [i] The install log is located at: /etc/pihole/install.log
  [✓] Installation complete!
ssuther6@pihole:~ $
```

Routing Network Traffic Through Pi-hole

Normally you would go into your router settings and change the DNS server to the Pi's IP address. The Verizon router does not let you change the WAN DNS through its interface, so I used a different approach.

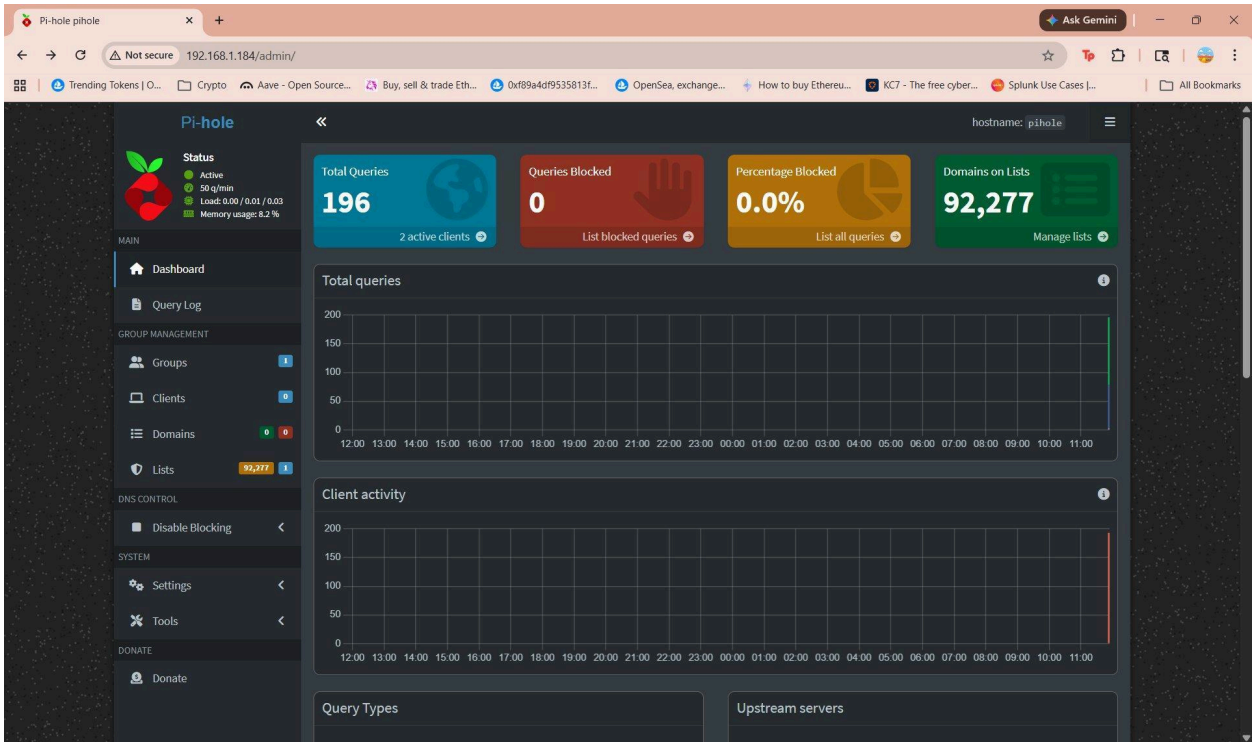
Pi-hole has its own DHCP server built in. I disabled the router's DHCP server and turned on Pi-hole's DHCP instead. Now when any device connects to the network, the Pi hands it an IP address and tells it to use itself for DNS. Every device is covered automatically with no extra configuration on each one.

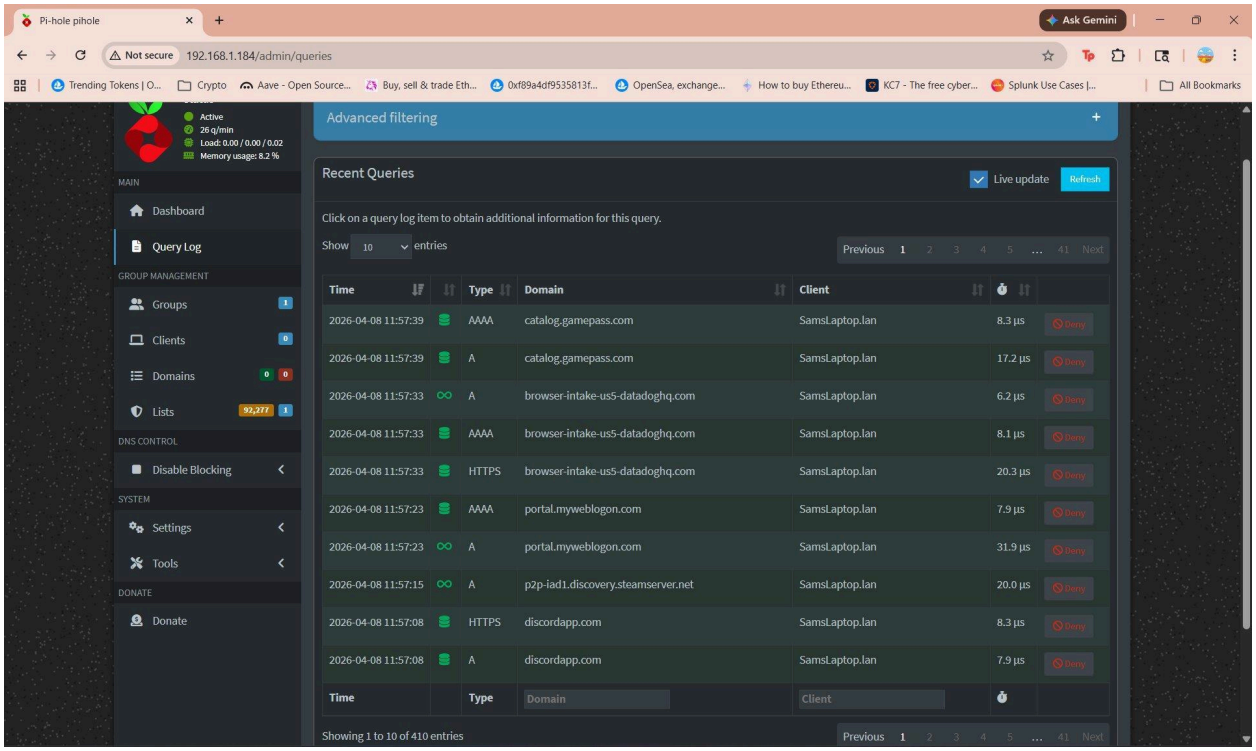


Verification

To confirm DNS was routing through Pi-hole, I ran nslookup google.com from my laptop. The server came back as pi.hole at 192.168.1.184. That confirmed it was working. Opening the web dashboard showed hundreds of queries already being logged across both active clients.

```
PS C:\Users\sampa> nslookup google.com
Server:  pi.hole
Address:  192.168.1.184

Non-authoritative answer:
Name:    google.com
Addresses:  2607:f8b0:4024:c09::65
          2607:f8b0:4024:c09::64
          2607:f8b0:4024:c09::8b
          2607:f8b0:4024:c09::66
          142.251.117.139
          142.251.117.113
          142.251.117.101
          142.251.117.102
          142.251.117.138
          142.251.117.100

PS C:\Users\sampa>
```





What Went Wrong

The Pi did not connect to WiFi on the first flash attempt, which meant pulling the SD card and starting over. I also had to re-flash a second time after switching from the Spectrum router to the Verizon router, since the Pi needed to be on the same network I was configuring. Both times were good practice with the setup process.

After the second flash I got a warning in SSH that looked like a security alert, saying the host key had changed and someone might be eavesdropping. It is not an attack. It happens because re-flashing the SD card generates a new host key, so SSH does not recognize the Pi anymore. The fix is running ssh-keygen -R pihole.local to clear the old key, then connecting again.

ISP routers are more locked down than I expected. Both Spectrum and Verizon restrict what you can change through their web interfaces. The DHCP takeover method works around this cleanly, but it took some time to figure out the right approach.

Results

Pi-hole is fully running on the home network with 92,277 domains on the blocklist. Every device connected to the router gets its DNS through the Pi automatically. The query log shows all traffic in real time: every device, every domain it tried to reach, and whether it was allowed or blocked. Watching that come in live makes it obvious how much background traffic devices generate constantly, even when you are not actively using them.

Skills Practiced

- Linux CLI and SSH remote administration
- DNS fundamentals and how it interacts with DHCP
- Network configuration and troubleshooting on real hardware
- Working around ISP router restrictions using DHCP takeover
- SSH host key verification and handling key changes after re-imaging